LIBRARY MANAGEMENT WITH BOUNTY SYSTEM

Project work submitted to bharathiar university, Coimbatore

in Partial Fulfillment of the Requirements

for the Award of the Degree of

Bachelor of Science in Computer Science

Submitted by

DHARSHINI A

[Register Number: 2322K2339]

Under the guidance of

Mrs.S.KALAISELVI M.Sc., B.Ed., M.Phil

Assistant Professor in Department of Computer Science

Adharsh Vidhyalaya College of Arts and Science for Women

DEPARTMENT OF COMPUTER SCIENCE

ADHARSH VIDHYALAYA COLLEGE OF ARTS AND SCIENCE FOR WOMEN

[Affiliated to Bharathiar University]

PARUVACHI PO, JAMBAI VIA, BHAVANI TK, ERODE DT-638312

MARCH-2026

LIBRARY MANAGEMENT WITH BOUNTY SYSTEM

Bonafide work done by

Bachelor of Science in Computer Science

Submitted by

DHARSHINI A

[Register Number: 2322K2339]

Under the guidance of

Mrs.S.KALAISELVI M.Sc., B.Ed., M.Phil

Signature of the GUIDE **Signature of the HOD**

Submitted for the Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

CERTIFICATE

CERTIFICATE

This is to certify that the Project Report entitled **"LIBRARY MANAGEMENT WITH BOUNTY SYSTEM"** submitted to **Bharathiar University Coimbatore**, in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Science in Computer Science** is a record of original project work done by **Ms. DHARSHINI A (Register Number: 2322K2339)** during the period DECEMBER 2025– MARCH 2026 study in the **Department of Computer Science, Adharsh Vidhyalaya College of Arts and Science for Women, Paruvachi (PO), Jambai(VIA), Bhavani(TK), erode (DT),** under my supervision and guidance and the project report has not found the basics for the award or any Degree/ Diploma / Associate ship / Fellowship or any other similar title to any candidate or any University.

Head of the Departmen **Signature of the Guide**

Counter Signed

Principal

Adharsh Vidhyalaya College of arts & science for women

Paruvachi po, Jambai via,Bhavani tk, erode dt-638312

DECLARATION

DECLARATION

I, **DHARSHINI A** hereby declare that the project report entitled **"LIBRARY MANAGEMENT WITH BOUNTY SYSTEM",** submitted to the Bharathiar University in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Science in Computer Science** is a record of original and independent work carried out by me under the guidance Mrs.S KALAISELVI M.Sc., B.Ed., M.Phil., Head of the Department of Computer Science and supervision of the project guide in the Department of Computer Science, Adharsh Vidhyalaya College of Arts and Science for Women.I further declare that this project has not been submitted previously for the award of any Degree/ Diploma/ Associate ship/ Fellowship, or any other similar title in any University.

Signature of the Candidate

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

I extend my sincere thanks to **Thiru. 'Ideal' K.SIVALINGAM M.A., M.A., M.A.,** Chairman**,** Ideal and Adharsh Educational Institutions and **Dr.S.SELVAMANI SIVALINGAM M.A., M.Ed., M.Phil., Ph.D.,** Secretary and correspondent, Adharsh Educational Institutions to provide an excellent to study in this "Temple of Learning"

I extend my heartfelt gratitude to **Mr.V.PRABAKARAN M.C.A., M.Phil**., Principal (Incharge), Adharsh Vidhyalaya College of arts & science for women for his excellent support throughout my study in all the ways.

There is no word to express my feeling to represent my guide **Mrs.S.KALAISELVI M.Sc., B.Ed., M.Phil.,** Head of the Department of Computer Science, Adharsh Vidhyalaya College of Arts and Science for women, who is my inspiration and taught me a lot in selecting, processing, finalizing this project work. I sincere thank her dedicative guidance to complete my project work successfully.

It gives immense pleasure to me be grateful to the faculty of Computer Science Department and other Department, Non-teaching staffs of Adharsh Vidhyalaya College of Arts and Science for women for their moral and selfless support.

I would like to thanks my parents, Friends and Relative for their motivation in the completion of the project. Last but not least, I extend my sincere thanks that are all behind the successful completion of this project work.

DHARSHINI A

SYNOPSIS

SYNOPSIS

The project titled **"Library Management with Bounty System"** is a computerized application designed to automate and manage the core operations of a library in an efficient and organized manner. A standard Library Management System typically includes facilities such as maintaining book records, managing student information, issuing and returning books, tracking due dates, calculating fines for late returns, and generating reports for administrative purposes. These systems help reduce manual record keeping and improve the accuracy and speed of library operations.

Although modern library systems provide effective management of books and transactions, they generally do not address the situation where a required book is currently unavailable because it is already issued to another student. In such cases, students usually have to wait until the book is returned, which may delay their study or research work.

To address this limitation, the proposed system introduces a **Bounty Request Feature** as an innovative extension to the traditional library management system. Through this feature, a student can post a request when a required book is unavailable in the library. Other students who currently possess the book can view the request and claim it by returning the book earlier through the system. This mechanism improves book circulation and helps students access important study materials more quickly.

The system is developed using **Python** with a graphical user interface built using **Tkinter**, while the database is managed using SQL-based data storage. By combining the standard functionalities of a library management system with the additional bounty request feature, the project provides a more interactive and efficient solution for managing library resources in educational institutions.

CONTENTS

CONTENTS

S.No	Title	Page No
	ACKNOWLEDGEMENT	
	SYNOPSIS	
1	**INTRODUCTION**	**1**
	1.1 ORGANIZATION PROFILE	**2**
	1.2 SOFTWARE DESCRIPTION	**3**
	1.2.1 HARDWARE SPECIFICATION	**3**
	1.2.2 SOFTWARE SPECIFICATION	**3**
2	**SYSTEM STUDY**	**9**
	2.1 EXISTING SYSTEM	**9**
	2.1.1 DRAWBACKS OF EXISTING SYSTEM	**9**
	2.2 PROPOSED SYSTEM	**10**
	2.2.1 FEATURES OF PROPOSED SYSTEM	**10**
3	**SYSTEM DESIGN AND DEVELOPMENT**	**11**
	3.1 FILE DESIGN	**11**
	3.2 INPUT DESIGN	**11**
	3.3 OUTPUT DESIGN	**12**
	3.4 DATABASE DESIGN	**12**
	3.5 SYSTEM DEVELOPMENT	**13**
	3.5.1 DESCRIPTION OF MODULES	**13**
4	**SYSTEM TESTING AND IMPLEMENTATION**	**14**
5	**CONCLUSION**	**17**
	BIBLIOGRAPHY	**18**
	APPENDICES	**19**
	A. DATA FLOW DIAGRAM	**19**
	B. TABLE STRUCTURE	**21**
	C. SAMPLE CODING	**22**
	D. SAMPLE INPUT	**45**
	E. SAMPLE OUTPUT	**51**

INTRODUCTION

INTRODUCTION

ABOUT THE PROJECT

The **Library Management with Bounty System** is a computerized application developed to automate and improve the overall management of library activities. The main objective of this project is to replace the traditional manual record-keeping system with an efficient, secure, and user-friendly digital platform that enhances operational accuracy and accessibility.

In many Educational institutions, library operations such as maintaining book records, issuing and returning books, tracking due dates, and managing student information are often handled manually using registers. This manual process is time-consuming, prone to human errors, and difficult to manage when the number of books and users increases. As the library grows, searching for book details, tracking issued books, and managing records becomes increasingly complex.

To overcome these limitations, the proposed system introduces an automated library management solution developed using modern programming tools. The system allows administrators to efficiently manage book records, student details, book issuance, returns, and fine calculations in a structured digital environment.

A key feature of this project is the **Bounty Request System**. When a student needs a book that is currently unavailable in the library, the student can post a bounty request for that book. Other students who currently possess the book can view the request and claim it by returning the book earlier through the library system. This feature helps improve book availability, reduces waiting time for students, and encourages better resource sharing among library users.

Overall, the Library Management System with Bounty Request System improves efficiency, reduces manual workload, and provides a smarter and more interactive approach to managing library resources.

1.1 ORGANIZATION PROFILE

The project is carried out at Adharsh Vidhyalaya College of Arts and Science for Women under the academic framework of Bharathiar University as a final year project by Dharshini A**.** The institution encourages practical learning and supports the development of real-time applications that address problems within the campus environment. In this context, the Library Management System with Bounty System is designed to improve the efficiency of the college library by automating operations such as book management, student records, issue and return tracking, and fine calculation. Unlike existing systems that focus only on basic record management, this project introduces a bounty request mechanism where students can request unavailable books and other students can respond by returning them early, thereby improving book circulation and reducing waiting time. This system reduces manual workload for library staff and enhances student interaction, resource sharing, and accessibility of academic resources in an efficient and practical manner.

1.2 SYSTEM SPECIFICATION

System specification describes the hardware and software requirements needed to develop and operate the Library Management System effectively. The system is designed to work efficiently on standard computer systems without requiring high-end infrastructure.

1.2.1 HARDWARE SPECIFICATION

Processor : AMD Ryzen 3 7320U with Radeon Graohics

RAM : 8.00 GB

Hard Disk : 50.00 GB

Monitor : 15-inch or Above

Input Devices : Keyboard and Mouse

Output Devices : Monitor

.1.2.2 SOFTWARE SPECIFICATION

Operating System : Windows 11

Programming Language : Python

GUI Framework : Tkinter

Database : SQL

Development Tool : Visual Studio Code

ABOUT THE SOFTWARE

An operating system is the core system software that manages computer hardware and software resources and provides a platform for running application programs. It acts as an interface between the user and the computer system.

In this project, Windows 11 is used as the operating system, which offers a modern desktop environment with improved performance, security, and user experience. It allows efficient execution of development tools like Visual Studio Code and supports programming languages such as Python and Visual Basic. The desktop interface makes it easy to manage files, run applications, and perform multitasking efficiently.

FEATURES OF WINDOWS 11

Windows 11 is a modern operating system developed by Microsoft, designed to provide a smooth, secure, and user-friendly experience. It includes several advanced features that improve performance, multitasking, and overall usability, making it suitable for both general use and software development.

FEATURES:

- User-friendly and modern graphical interface
- Centered Start Menu and redesigned taskbar
- Improved performance and faster processing speed
- Enhanced security features like Windows Defender and Secure Boot
- Advanced multitasking with Snap Layouts and Snap Groups
- Better memory and resource management
- Virtual desktops for organized workspace
- Improved support for latest software and hardware
- Seamless updates and system stability

VISUAL STUDIO CODE

Visual Studio Code is a lightweight yet powerful source code editor developed by Microsoft. It is widely used by developers for writing, editing, and debugging code efficiently. It provides a flexible development environment with support for multiple programming languages and tools, making it highly suitable for building modern applications.

FEATURES OF VISUAL STUDIO CODE:

- Lightweight and fast performance
- Supports multiple programming languages (Python, C, Java, etc.)
- Built-in terminal for executing commands
- Extension support for adding new features
- Debugging tools for error detection and fixing
- IntelliSense for smart code completion and suggestions
- Git integration for version control
- Customizable interface and themes
- Auto-save and syntax highlighting

SYSTEM STUDY

SYSTEM STUDY

2.1 EXISTING SYSTEM

In modern Educational institutions, many libraries already use computerized Library Management Systems to manage their daily operations. These systems allow librarians to maintain digital records of books, students, book issuing, and return transactions. Popular library software solutions provide features such as book catalog management, automated issue and return tracking, fine calculation, and report generation.

In the existing digital systems, librarians can store book details including title, author, category, and quantity in a centralized database. Student records are also maintained digitally, allowing quick access to borrowing history and transaction details. When a book is issued, the system automatically records the issue date, due date, and return information.

Most current systems also provide search functionality to locate books quickly and generate reports such as overdue books, issued books list, and student borrowing history. These systems significantly reduce manual work and improve operational efficiency compared to traditional paper-based systems.

However, even though these systems automate basic library operations, they mainly focus on **record management and transaction processing**. They do not provide intelligent mechanisms to improve book availability when a required book is currently issued to another student.

When a book is unavailable, students usually have to wait until the current borrower returns it. There is no built-in feature in most library systems that allows students to request urgent access to a book or communicate their need to other borrowers.

2.2 DRAWBACKS OF EXISTING SYSTEM

In modern educational institutions, many libraries use computerized Library Management Systems to handle daily operations. These systems help in maintaining digital records of books, students, and transactions efficiently. They reduce manual work and improve accuracy compared to traditional methods. However, despite these advantages, existing systems still have several limitations.

Drawbacks:

- **No Solution for Book Unavailability**
  When a book is issued, other students must wait until it is returned, as there is no alternative mechanism to access it.
- **Lack of Communication Between Users**
  Existing systems do not allow students to communicate their need for a book to other borrowers.
- **No Priority-Based Access System**
  There is no feature to handle urgent requests or prioritize students who need a book immediately.
- **Inefficient Book Circulation**
  Books remain with borrowers until the due date, even if other students urgently require them.
- **Absence of Smart Features**
  Current systems lack intelligent functionalities that can improve availability and usage of books.
- **Delay in Accessing Required Books**
  Students often face delays in getting important books, which can affect their studies.

2.3 PROPOSED SYSTEM

To address the limitations of existing systems, the proposed **Library Management with Bounty System** introduces an enhanced digital solution that not only automates library operations but also improves book accessibility.

The system manages book records, student records, book issuing, returns, fine calculation, and report generation in a structured digital environment. The system provides a graphical user interface that is easy to operate and improves efficiency for library administrators.

The major innovation introduced in this project is the **Bounty Request System**. When a student needs a book that is currently unavailable in the library, the student can create a bounty request within the system specifying the required book.

Other students who currently possess the book can view these requests. If they are willing to return the book earlier through the library system, they can claim the bounty request. Once the book becomes available and is issued to the requesting student, the request is marked as completed.

This approach improves book circulation, reduces waiting time for students, and introduces a collaborative method of resource sharing within the library.

2.4 FEATURES OF PROPOSED SYSTEM

The proposed system introduces an innovative **Bounty Request System** to improve book availability in the library. When a required book is not available in stock, students do not have to wait until it is returned. Instead, the system allows them to request the book through a bounty mechanism. This approach improves book circulation, reduces waiting time, and provides a smarter way to access resources.

Features:

- **Bounty Request Creation**
  Students can place a bounty request when a required book is currently unavailable in the library.
- **Visibility to Other Students**
  The request is visible to students who have already borrowed that book.
- **Early Return Option**
  Students holding the book can choose to return it earlier by claiming the bounty request.
- **Improved Book Circulation**
  Books are circulated more efficiently instead of being held until the due date.
- **Reduced Waiting Time**
  Students can access important books faster without waiting for the normal return period.
- **Bounty Status Tracking**
  Each request is marked as *Open* when created and changes to *Completed* once the book is returned.
- **User-Friendly Interaction**
  The system provides a simple and clear way for students to request and respond to book needs.

SYSTEM DESIGN AND DEVELOPMENT

SYSTEM DESIGN AND DEVELOPMENT

3.1 FILE DESIGN

File design refers to the logical structure used to store and manage system data. In this project, data is stored using **SQL** database tables which ensure efficient storage and quick retrieval of records.

Each file (table) stores specific information such as book details, student information, issue transactions, return transactions, fine records, and bounty requests.

Primary keys are used to uniquely identify records in each table, while relationships between tables maintain consistency and prevent redundancy. The file design ensures that data can be easily inserted, updated, deleted, and retrieved without affecting other system operations.

3.2 INPUT DESIGN

Input design focuses on collecting accurate data from users through a user-friendly interface. The system provides various input forms such as book entry form, student registration form, issue book form, and return book form.

Validation techniques are applied to ensure correct data entry. The system checks mandatory fields, prevents duplicate entries, and ensures proper format for numerical and text inputs. The input design reduces user errors and improves data accuracy.

3.3 OUTPUT DESIGN

Output design presents processed information in a clear, structured, and user-friendly format. The system displays outputs such as book availability, student records, issued/returned books, error display, and bounty status. These outputs help users easily monitor library activities and track transactions.All results are shown through a Python Tkinter-based graphical interface for better readability. Data is retrieved using SQL queries and can also be printed for record-keeping.

3.4 DATABASE DESIGN

The database is designed using **SQL** to store and manage system data efficiently. The database contains multiple tables such as:

• **Books Table** – Stores book details including book ID, title, author, category, and quantity.
• **Students Table** – Stores student details including student ID, name, department, and contact information.
• **Transactions Table** – Stores issue and return details including issue date, due date, and return date.

• **Bounty Table** – Stores bounty requests including book title, requester ID, claim status, and student who claimed the request.

3.5 SYSTEM DEVELOPMENT

The system is developed using Python programming language with Tkinter for graphical user interface design. The application follows modular programming methodology, where each module is developed separately and later integrated into the main system.

Python libraries such as SQLite3 are used for database connectivity. Additional libraries like Pillow are used for image handling, while smtplib supports notification features. Threading is used to improve system performance by allowing background processing.

The development process includes planning, designing, coding, debugging, and testing to ensure proper functionality.

3.6 DESCRIPTION OF MODULES

Core Functionalities of the System

The Library Management System performs the following major operations:

1. Book Management

- Add new books to the database
- Update book details (title, author, publisher, category, quantity)
- Delete book records
- Search books by title, author, or ID
- Track available and issued quantity

2. Student/User Management

- Register new students
- Update student details

- Delete student records
- Maintain borrowing history for each student

3. Book Issue Module

- Issue books to registered students
- Automatically record issue date
- Set return deadline
- Prevent issuing if book stock is unavailable
- Prevent issuing if student exceeds borrowing limit

4. Book Return Module

- Record return date
- Automatically calculate fine for late return
- Update book stock
- Maintain transaction history

Unique Feature: Bounty System (Innovative Enhancement)

Book Bounty Claim System

The Book Bounty Claim System is an innovative feature introduced to improve book availability and student collaboration.

In many cases, students require books urgently for academic purposes, but the book may already be issued to another student. To solve this problem, the system introduces a bounty request mechanism.

Working of Book Bounty Claim System

1. **Posting Bounty Request**
    - If a required book is not available in the library, a student can post a bounty request.
    - The request contains book name, student ID, and urgency details.
2. **Viewing Bounty Requests**
    - All bounty requests are displayed in the system.
    - Students who currently possess the requested book can view these requests.
3. **Claiming the Bounty**
    - A student who has the book can claim the bounty request.
    - The claiming student can return the book earlier or transfer it through library approval.
4. **Completion of Request**
    - Once the book becomes available and is issued to the requesting student, the bounty request is marked as completed.

SYSTEM TESTING AND IMPLEMENTATION

SYSTEM TESTING AND IMPLEMENTATION

4.1 SYSTEM TESTING

System testing is performed to ensure that the application works according to the requirements. Various testing methods are used including unit testing, integration testing, and user acceptance testing.

Unit Testing

Unit testing is performed on individual modules of the system to ensure each part works correctly. In this project, modules such as add book, add student, issue book, return book, and bounty posting are tested separately.

Points:

- Tested book management functions (add, delete, view)
- Verified student registration and data storage
- Checked issue and return functions individually
- Tested bounty posting and claiming separately
- Ensured each function executes without errors

Integration Testing

Integration testing is conducted to ensure that all modules work together properly. It verifies that data flows correctly between GUI forms, backend logic, and the SQLite database.

Points:

- Checked interaction between book issue and database update
- Verified return module updates book quantity correctly
- Ensured bounty system links with issue/return process

- Tested combined execution of multiple modules
- Confirmed smooth data flow between components

Validation Testing

Validation testing ensures that the system meets user requirements and handles inputs correctly. It checks whether the system accepts valid data and rejects invalid inputs.

Points:

- Validated input fields (student ID, book ID, etc.)
- Prevented issuing books when stock is zero
- Restricted invalid or empty inputs
- Ensured correct calculation of due dates and fines
- Verified correct status update in bounty system

White Box Testing

White box testing is performed by analyzing the internal code logic of the system. The program flow, conditions, loops, and database queries are tested to ensure correctness.

Points:

- Verified Python logic and conditions in functions
- Checked loops and control statements
- Tested SQL queries execution
- Ensured proper error handling in code
- Improved code efficiency and correctness

Black Box Testing

Black box testing is performed by testing the system from the user's perspective without focusing on internal code. It ensures that the system produces correct output for given inputs.

Points:

- Tested GUI forms for correct output
- Checked user actions like issue, return, bounty claim
- Verified system response for valid and invalid inputs
- Ensured proper display of messages and results
- Confirmed overall functionality from user side

4.2 SYSTEM IMPLEMENTATION

System implementation involves installing and running the application in a real-time environment. The software is installed on computers in the library, and necessary database configurations are completed.

Training is provided to users to help them operate the system effectively. The implementation process ensures smooth transition from manual system to computerized system. The system improves operational efficiency and reduces workload for librarians. The application is installed on the system, and the SQLite database is configured properly. The system is then made available for use by librarians and students through a user-friendly interface.

Points:

- Installed Python environment and required libraries
- Configured SQLite database for storing records
- Deployed Tkinter-based GUI application
- Tested system in real-time usage environment
- Ensured smooth transition from manual to digital system

CONCLUSION

CONCLUSION

The **Library Management with Bounty System** successfully automates library operations and improves efficiency, accuracy, and reliability. The system replaces manual record keeping with a digital platform that simplifies book management, student tracking, book issuing and returning, fine calculation, and report generation.

The introduction of the **Bounty Request System** provides an innovative solution for situations when a required book is unavailable in the library. Students can request a book through the system, and other students who currently possess the book can claim the request by returning it earlier. This feature improves book circulation and reduces waiting time for students.

The project demonstrates the practical application of **Python programming, database management using SQL queries, and graphical user interface development with Tkinter. Overall, the system provides an efficient and scalable solution for managing library resources in educational institutions.

Future Enhancement

The system can be further enhanced by introducing a **Smart Bounty Matching system using AI**, which automatically connects users requesting a book with those who have it available, improving efficiency and reducing search time. Additionally, a **Peer-to-Peer Lending feature** can be implemented, allowing students to lend their personal books through the platform in exchange for bounty rewards. In cases where multiple students claim to provide the same requested book, a **priority-based allocation mechanism** can be introduced. This mechanism can decide the provider based on factors such as who responded first, user reliability score, past contributions, or proximity. These enhancements will make the system more intelligent, fair, and user-focused, while strengthening the uniqueness of the bounty-based approach.

BIBLIOGRAPHY

BIBLIOGRAPHY

Books:

1. **"Python Crash Course"** – Eric Matthes, No Starch Press
2. **"Learning Python"** – Mark Lutz, O'Reilly Media
3. **"Automate the Boring Stuff with Python"** – Al Sweigart, No Starch Press
4. **"Head First Python"** – Paul Barry, O'Reilly Media
5. **"Database System Concepts"** – Abraham Silberschatz, Henry F. Korth, S. Sudarshan, McGraw-Hill
6. **"SQL for Beginners"** – Allen G. Taylor, Wiley Publications
7. **"Software Engineering: A Practitioner's Approach"** – Roger S. Pressman, McGraw-Hill
8. **"Fundamentals of Database Systems"** – Ramez Elmasri, Shamkant B. Navathe, Pearson

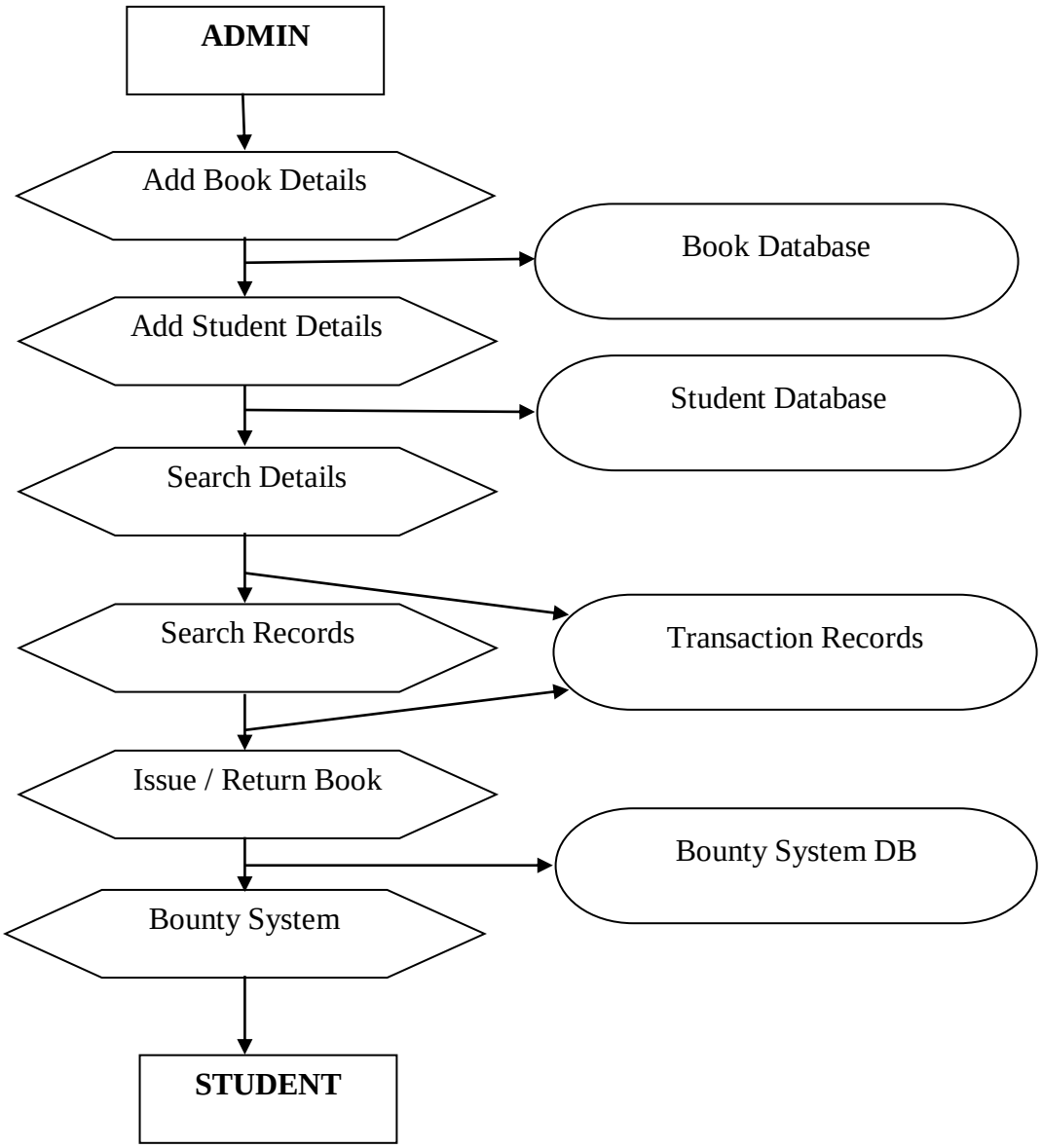
Web References:

1. Python Official Documentation – https://docs.python.org
2. SQLite Official Documentation – https://www.sqlite.org/docs.html
3. Tkinter Documentation – https://docs.python.org/3/library/tkinter.html
4. Visual Studio Code Documentation – https://code.visualstudio.com/docs
5. W3Schools Python Tutorial – https://www.w3schools.com/python/
6. GeeksforGeeks (Python & SQL) – https://www.geeksforgeeks.org/

APPENDICES

APPENDICES

A. Data Flow Diagram



B. Table Structure

1. Books Table

Field Name	Field Type	Size	Constraint	Description
Book_ID	Integer	10	Primary Key	Unique ID of the book
Title	Text	100	Not Null	Name of the book
Author	Text	100	Not Null	Author of the book
Category	Text	50	Not Null	Subject/category of the book
Quantity	Integer	5	Not Null	Total number of books available

This table stores information about all books available in the library.

2. Students Table

Field Name	Field Type	Size	Constraint	Description
Student_ID	Integer	10	Primary Key	Unique student ID
Name	Text	100	Not Null	Student name
Email	Text	100	Not Null	Student email address

This table stores details of students who are registered in the library system.

3. Table Name: Transactions

Field Name	Field Type	Size	Constraint	Description
Transaction_ID	Integer	10	Primary Key	Unique transaction ID
Book_ID	Integer	10	Foreign Key	References Books table
Student_ID	Integer	10	Foreign Key	References Students table
Issue_Date	Date	—	Not Null	Date when book was issued
Due_Date	Date	—	Not Null	Expected return date
Return_Date	Date	—	Null Allowed	Actual return date

4. Table Name: Bounties

Field Name	Field Type	Size	Constraint	Description
Bounty_ID	Integer	10	Primary Key	Unique bounty request ID
Book_Title	Text	100	Not Null	Title of the requested book
Requester_ID	Integer	10	Foreign Key	Student who posted request
Message	Text	100	Null Allowed	Request message
Status	Text	20	Not Null	Open / Claimed / Completed
Claimed_By	Integer	10	Foreign Key	Student who claimed request

5. Table Name: Policies

Field Name	Field Type	Size	Constraint	Description
Policy_ID	Integer	10	Primary Key	Unique policy identifier
Category	Text	50	Not Null	Book category
Max_Books	Integer	5	Not Null	Maximum books allowed
Issue_Days	Integer	5	Not Null	Number of days allowed
Fine_Rate	Integer	5	Not Null	Fine per late day

C. Sample Coding

```
import sqlite3
from tkinter import *
from tkinter import messagebox, ttk
from datetime import date, timedelta
from PIL import Image, ImageTk
import os
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
import threading

# ---------------- DATABASE ----------------
conn = sqlite3.connect("library.db")
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS books(
    book_id INTEGER PRIMARY KEY,
    title TEXT,
    author TEXT,
    quantity INTEGER,
    category TEXT,
    reference INTEGER DEFAULT 0
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS students(
    student_id INTEGER PRIMARY KEY,
```

```
    name TEXT,
    email TEXT
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS issues(
    issue_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER,
    student_id INTEGER,
    issue_date TEXT,
    due_date TEXT,
    return_date TEXT,
    status TEXT DEFAULT 'Not Returned'
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS bounties(
    bounty_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_title TEXT,
    requester_id INTEGER,
    reward TEXT,
    status TEXT DEFAULT 'Open',
    claimed_by INTEGER
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS policies(
    category TEXT PRIMARY KEY,
```

```
    max_books INTEGER,
    issue_days INTEGER,
    fine_rate REAL
)
""")

cur.execute("INSERT OR IGNORE INTO policies VALUES ('General', 3, 14, 1.0)")
cur.execute("INSERT OR IGNORE INTO policies VALUES ('Reference', 1, 0, 0.0)")
conn.commit()

# ---------------- EMAIL FUNCTION ----------------
def send_email_to_students(book_title, book_author):
    try:
        # --- NEW CONNECTION INSIDE THREAD ---
        conn_thread = sqlite3.connect("library.db")
        cur_thread = conn_thread.cursor()

        sender_email = "dharshinike20@gmail.com"  # your email
        sender_password = "wjujjlsdwedrjbfo"     # App password without spaces

        cur_thread.execute("SELECT email, name FROM students")
        students = cur_thread.fetchall()

        if not students:
            print("No students to send email to.")
            conn_thread.close()
            return

        subject = f"New Book Added: {book_title}"
        for student in students:
            receiver_email, student_name = student
```

```
        msg = MIMEMultipart()
        msg['From'] = sender_email
        msg['To'] = receiver_email
        msg['Subject'] = subject

        body = f"Hello {student_name},\n\nA new book '{book_title}' by {book_author} has
been added to the library. Check it out!\n\n- Library Team"
        msg.attach(MIMEText(body, 'plain'))

        try:
            with smtplib.SMTP('smtp.gmail.com', 587) as server:
                server.starttls()
                server.login(sender_email, sender_password)
                server.sendmail(sender_email, receiver_email, msg.as_string())
            print(f"Email sent to {receiver_email}")
        except Exception as e:
            print(f"Error sending email to {receiver_email}: {e}")

    conn_thread.close()
  except Exception as e:
    print("Email thread error:", e)

# ---------------- GUI ----------------
root = Tk()
root.title("Advanced Policy-Driven Library System")
root.geometry("900x700")

# ---------------- BACKGROUND IMAGE ----------------
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
BG_PATH = os.path.join(BASE_DIR, "library_bg.png")
```

```
if os.path.exists(BG_PATH):
    bg_img = Image.open(BG_PATH)
    bg_img = bg_img.resize((1500, 900))
    bg_photo = ImageTk.PhotoImage(bg_img)
    bg_label = Label(root, image=bg_photo)
    bg_label.place(x=0, y=0, relwidth=1, relheight=1)
    # -------- DARK OVERLAY FOR ELEGANCE --------
overlay = Frame(root, bg="#2B1B17")   # deep chocolate brown
overlay.place(relwidth=1, relheight=1)

overlay.lift()      # bring overlay above bg
overlay.lower()     # keep bg at bottom

# ---------------- MAIN TITLE ----------------
title_label = Label(
    root,
    text="LIBRARY\nMANAGEMENT SYSTEM",
    font=("Georgia", 36, "bold"),
    bg="#2B1B17",
    fg="#F5E6D3",
    justify="center"
)
title_label.place(relx=0.5, rely=0.12, anchor="center")

# ---------------- STYLISH BUTTON STYLE ----------------
def create_style():
    style = ttk.Style()
    style.theme_use("alt")

    style.configure(
        "Custom.TButton",
```

```
        font=("Segoe UI Semibold", 12),
        padding=10,
        background="#3E2723",   # dark brown
        foreground="#F5E6D3",   # soft cream
        borderwidth=0
    )

    style.map(
        "Custom.TButton",
        background=[
            ("active", "#5D4037"),   # lighter brown on hover
            ("pressed", "#2B1B17")
        ]
    )

    return style

# ---------------- BOOK ----------------
def add_book():
    def submit():
        try:
            ref = 1 if ref_var.get() else 0
            cur.execute(
                "INSERT INTO books VALUES (?,?,?,?,?,?)",
                (int(bid.get()), title.get(), author.get(), int(qty.get()), category.get(), ref)
            )
            conn.commit()
            messagebox.showinfo("Success", f"Book '{title.get()}' added.")

            # Send emails in background
```

```
            threading.Thread(target=send_email_to_students, args=(title.get(), author.get()),
daemon=True).start()

            win.destroy()
        except Exception as e:
            messagebox.showerror("Error", e)

    win = Toplevel(root)
    win.title("Add Book")
    win.geometry("400x350")
    win.resizable(False, False)

    # Center window
    win.transient(root)
    win.grab_set()

    # Table-style frame
    frame = Frame(win, padx=20, pady=20)
    frame.pack(expand=True, fill=BOTH)

    # Table headers
    headers = ["Field", "Value"]
    for i, header in enumerate(headers):
        Label(frame, text=header, font=('Segoe UI', 10, 'bold'), bg='#f0f0f0', relief='solid',
bd=1, padx=10, pady=8, width=15).grid(row=0, column=i, sticky='ew', pady=2)

    # Table rows
    Label(frame, text="Book ID", font=('Segoe UI', 9)).grid(row=1, column=0, sticky='w',
pady=5)
    bid = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    bid.grid(row=1, column=1, sticky='ew', pady=5, padx=(5,0))
```

```
    Label(frame, text="Title", font=('Segoe UI', 9)).grid(row=2, column=0, sticky='w',
pady=5)
    title = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    title.grid(row=2, column=1, sticky='ew', pady=5, padx=(5,0))

    Label(frame, text="Author", font=('Segoe UI', 9)).grid(row=3, column=0, sticky='w',
pady=5)
    author = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    author.grid(row=3, column=1, sticky='ew', pady=5, padx=(5,0))

    Label(frame, text="Quantity", font=('Segoe UI', 9)).grid(row=4, column=0, sticky='w',
pady=5)
    qty = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    qty.grid(row=4, column=1, sticky='ew', pady=5, padx=(5,0))

    Label(frame, text="Category", font=('Segoe UI', 9)).grid(row=5, column=0, sticky='w',
pady=5)
    category = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    category.grid(row=5, column=1, sticky='ew', pady=5, padx=(5,0))

    ref_var = IntVar()
    Checkbutton(frame, text="Reference Book (Cannot be issued)", variable=ref_var,
font=('Segoe UI', 9)).grid(row=6, column=0, columnspan=2, sticky='w', pady=5)

    # Configure column weights
    frame.columnconfigure(1, weight=1)

    style = create_style()
    ttk.Button(frame, text="Add Book", style='Custom.TButton',
command=submit).grid(row=7, column=0, columnspan=2, pady=20)
```

```
from tkinter import ttk

def view_books():
    win = Toplevel(root)
    win.title("Books")
    win.geometry("900x400")
    win.configure(bg="#F4F6F9")

    style = ttk.Style()
    style.theme_use("default")

    style.configure("Treeview",
                    background="white",
                    foreground="#111827",
                    rowheight=25,
                    fieldbackground="white")

    style.configure("Treeview.Heading",
                    background="#1F2933",
                    foreground="white",
                    font=('Segoe UI', 10, 'bold'))

    tree = ttk.Treeview(win,
columns=("ID","Title","Author","Qty","Category","Reference"), show="headings")

    for col in tree["columns"]:
        tree.heading(col, text=col)
        tree.column(col, width=130)

    tree.pack(fill=BOTH, expand=True)
```

```
    cur.execute("SELECT * FROM books")
    rows = cur.fetchall()

    for index, b in enumerate(rows):
        ref_status = "Yes" if b[5] else "No"
        tree.insert("", END, values=(b[0], b[1], b[2], b[3], b[4], ref_status))


def delete_book():
    def submit():
        bid_v = bid.get()
        cur.execute("SELECT * FROM books WHERE book_id=?", (bid_v,))
        if not cur.fetchone():
            messagebox.showerror("Error", "Book ID not found")
            return
        cur.execute("DELETE FROM books WHERE book_id=?", (bid_v,))
        conn.commit()
        messagebox.showinfo("Deleted", "Book deleted successfully")
        win.destroy()

    win = Toplevel(root)
    win.title("Delete Book")
    win.geometry("350x200")
    win.resizable(False, False)
    win.transient(root)
    win.grab_set()

    frame = Frame(win, padx=20, pady=20)
```

```
    Label(frame, text="Book ID", font=('Segoe UI', 9)).grid(row=1, column=0, sticky='w',
pady=20)
    bid = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    bid.grid(row=1, column=1, sticky='ew', pady=20, padx=(5,0))

    frame.columnconfigure(1, weight=1)

    style = create_style()
    ttk.Button(frame, text="Delete Book", style='Custom.TButton',
command=submit).grid(row=2, column=0, columnspan=2, pady=10)

# ---------------- STUDENT ----------------
def add_student():
    def submit():
        try:
            cur.execute(
                "INSERT INTO students VALUES (?,?,?)",
                (int(sid.get()), name.get(), email.get())
            )
            conn.commit()
            messagebox.showinfo("Success", "Student added")
            win.destroy()
        except Exception as e:
            messagebox.showerror("Error", e)

    win = Toplevel(root)
    win.title("Add Student")
    win.geometry("400x300")
    win.resizable(False, False)
    win.transient(root)
    win.grab_set()
```

```
    frame = Frame(win, padx=20, pady=20)
    frame.pack(expand=True, fill=BOTH)

    headers = ["Field", "Value"]
    for i, header in enumerate(headers):
        Label(frame, text=header, font=('Segoe UI', 10, 'bold'), bg='#f0f0f0', relief='solid',
bd=1, padx=10, pady=8, width=15).grid(row=0, column=i, sticky='ew', pady=2)

    Label(frame, text="Student ID", font=('Segoe UI', 9)).grid(row=1, column=0, sticky='w',
pady=5)
    sid = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    sid.grid(row=1, column=1, sticky='ew', pady=5, padx=(5,0))

    Label(frame, text="Name", font=('Segoe UI', 9)).grid(row=2, column=0, sticky='w',
pady=5)
    name = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    name.grid(row=2, column=1, sticky='ew', pady=5, padx=(5,0))

    Label(frame, text="Email", font=('Segoe UI', 9)).grid(row=3, column=0, sticky='w',
pady=5)
    email = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    email.grid(row=3, column=1, sticky='ew', pady=5, padx=(5,0))

    frame.columnconfigure(1, weight=1)

    style = create_style()
    ttk.Button(frame, text="Add Student", style='Custom.TButton',
command=submit).grid(row=4, column=0, columnspan=2, pady=20)

def view_students():
```

```
    win = Toplevel(root)
    win.title("Students")
    win.geometry("700x400")
    win.configure(bg="#F4F6F9")

    tree = ttk.Treeview(win, columns=("ID","Name","Email"), show="headings")

    tree.heading("ID", text="ID")
    tree.heading("Name", text="Name")
    tree.heading("Email", text="Email")

    tree.column("ID", width=100)
    tree.column("Name", width=200)
    tree.column("Email", width=300)

    tree.pack(fill=BOTH, expand=True)

    cur.execute("SELECT * FROM students")
    for s in cur.fetchall():
        tree.insert("", END, values=s)


def delete_student():
    def submit():
        sid_v = sid.get()
        cur.execute("SELECT * FROM students WHERE student_id=?", (sid_v,))
        if not cur.fetchone():
            messagebox.showerror("Error", "Student ID not found")
            return
        cur.execute("DELETE FROM students WHERE student_id=?", (sid_v,))
        conn.commit()
```

```
        messagebox.showinfo("Deleted", "Student deleted successfully")
        win.destroy()

    win = Toplevel(root)
    win.title("Delete Student")
    win.geometry("350x200")
    win.resizable(False, False)
    win.transient(root)
    win.grab_set()

    frame = Frame(win, padx=20, pady=20)
    frame.pack(expand=True, fill=BOTH)

    headers = ["Field", "Value"]
    for i, header in enumerate(headers):
        Label(frame, text=header, font=('Segoe UI', 10, 'bold'), bg='#f0f0f0', relief='solid',
bd=1, padx=10, pady=8, width=15).grid(row=0, column=i, sticky='ew', pady=2)

    Label(frame, text="Student ID", font=('Segoe UI', 9)).grid(row=1, column=0, sticky='w',
pady=20)
    sid = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    sid.grid(row=1, column=1, sticky='ew', pady=20, padx=(5,0))

    frame.columnconfigure(1, weight=1)

    style = create_style()
    ttk.Button(frame, text="Delete Student", style='Custom.TButton',
command=submit).grid(row=2, column=0, columnspan=2, pady=10)

# ---------------- ISSUE / RETURN ----------------
def issue_book():
```

```
    def submit():
        book_name = book.get().strip()
        sid_v = sid.get().strip()

        if not book_name or not sid_v.isdigit():
            messagebox.showerror("Error", "Enter valid Book Name and Student ID")
            return

        sid_v = int(sid_v)

        # Check student exists
        cur.execute("SELECT * FROM students WHERE student_id=?", (sid_v,))
        if not cur.fetchone():
            messagebox.showerror("Error", "Student ID not found")
            return

        # Check book exists by NAME
        cur.execute("SELECT * FROM books WHERE LOWER(title)=LOWER(?)",
(book_name,))
        book_data = cur.fetchone()

        if not book_data:
            messagebox.showwarning("Not Available", "Book not present in library")
            return

        book_id = book_data[0]
        quantity = book_data[3]
        category = book_data[4]
        is_reference = book_data[5]

        # Check stock
```

```
    if quantity == 0:
        messagebox.showwarning("Out of Stock", "Oops! Book is currently out of stock")
        return

    # Check reference book
    if is_reference == 1:
        messagebox.showwarning("Reference Book", "Cannot issue reference book")
        return

    # Policy check
    cur.execute("SELECT max_books, issue_days FROM policies WHERE category=?",
(category,))
    policy = cur.fetchone()

    if not policy:
        messagebox.showerror("Error", "Policy not defined for this category")
        return

    max_books, issue_days = policy

    cur.execute("""
        SELECT COUNT(*) FROM issues
        WHERE student_id=? AND return_date IS NULL
    """, (sid_v,))

    if cur.fetchone()[0] >= max_books:
        messagebox.showwarning("Limit Exceeded", "Maximum book issue limit
reached")
        return

    # Issue book
```

```
        due_date = date.today() + timedelta(days=issue_days)

        cur.execute("UPDATE books SET quantity=quantity-1 WHERE book_id=?",
(book_id,))
        cur.execute("""
            INSERT INTO issues (book_id, student_id, issue_date, due_date)
            VALUES (?,?,?,?)
        """, (book_id, sid_v, date.today(), due_date))

        conn.commit()
        messagebox.showinfo("Issued", f"Book issued successfully!\nDue Date: {due_date}")
        win.destroy()

    win = Toplevel(root)
    win.title("Issue Book")
    win.geometry("400x250")
    win.resizable(False, False)
    win.transient(root)
    win.grab_set()

    frame = Frame(win, padx=20, pady=20)
    frame.pack(expand=True, fill=BOTH)

    headers = ["Field", "Value"]
    for i, header in enumerate(headers):
        Label(frame, text=header, font=('Segoe UI', 10, 'bold'), bg='#f0f0f0', relief='solid',
bd=1, padx=10, pady=8, width=15).grid(row=0, column=i, sticky='ew', pady=2)

    Label(frame, text="Book Name", font=('Segoe UI', 9)).grid(row=1, column=0,
sticky='w', pady=10)
    book = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
```

```
    book.grid(row=1, column=1, sticky='ew', pady=10, padx=(5,0))

    Label(frame, text="Student ID", font=('Segoe UI', 9)).grid(row=2, column=0, sticky='w',
pady=10)
    sid = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    sid.grid(row=2, column=1, sticky='ew', pady=10, padx=(5,0))

    frame.columnconfigure(1, weight=1)

    style = create_style()
    ttk.Button(frame, text="Issue", style='Custom.TButton', command=submit).grid(row=3,
column=0, columnspan=2, pady=20)

def return_book():
    def submit():
        bid_v = int(bid.get())
        sid_v = int(sid.get())

        cur.execute("""
        SELECT issue_id FROM issues
        WHERE book_id=? AND student_id=? AND return_date IS NULL
        """, (bid_v, sid_v))
        issue = cur.fetchone()
        if issue:
            cur.execute("UPDATE issues SET return_date=?, status='Returned' WHERE
issue_id=?",
                        (date.today(), issue[0]))
            cur.execute("UPDATE books SET quantity=quantity+1 WHERE book_id=?",
(bid_v,))
            conn.commit()
            messagebox.showinfo("Returned", "Book returned successfully")
```

```
        win.destroy()
    else:
        messagebox.showwarning("Error", "No active issue found")

    win = Toplevel(root)
    win.title("Return Book")
    win.geometry("400x250")
    win.resizable(False, False)
    win.transient(root)
    win.grab_set()

    frame = Frame(win, padx=20, pady=20)
    frame.pack(expand=True, fill=BOTH)

    headers = ["Field", "Value"]
    for i, header in enumerate(headers):
        Label(frame, text=header, font=('Segoe UI', 10, 'bold'), bg='#f0f0f0', relief='solid',
bd=1, padx=10, pady=8, width=15).grid(row=0, column=i, sticky='ew', pady=2)

    Label(frame, text="Book ID", font=('Segoe UI', 9)).grid(row=1, column=0, sticky='w',
pady=10)
    bid = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    bid.grid(row=1, column=1, sticky='ew', pady=10, padx=(5,0))

    Label(frame, text="Student ID", font=('Segoe UI', 9)).grid(row=2, column=0, sticky='w',
pady=10)
    sid = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    sid.grid(row=2, column=1, sticky='ew', pady=10, padx=(5,0))

    frame.columnconfigure(1, weight=1)
```

```
    style = create_style()
    ttk.Button(frame, text="Return", style='Custom.TButton',
command=submit).grid(row=3, column=0, columnspan=2, pady=20)

# ====================================================
# ⋆⋆ BOUNTY SYSTEM ⋆⋆
# ====================================================

# POST BOUNTY
def post_bounty():
    def submit():
        try:
            cur.execute("""
            INSERT INTO bounties(book_title,requester_id,reward)
            VALUES (?,?,?)
            """,(book.get(),int(student.get()),reward.get()))

            conn.commit()
            messagebox.showinfo("Success","Bounty Posted")
            win.destroy()
        except Exception as e:
            messagebox.showerror("Error",e)

    win=Toplevel(root)
    win.title("Post Bounty")
    win.geometry("400x300")
    win.resizable(False, False)
    win.transient(root)
    win.grab_set()

    frame = Frame(win, padx=20, pady=20)
```

```
    frame.pack(expand=True, fill=BOTH)

    headers = ["Field", "Value"]
    for i, header in enumerate(headers):
        Label(frame, text=header, font=('Segoe UI', 10, 'bold'), bg='#f0f0f0', relief='solid',
bd=1, padx=10, pady=8, width=15).grid(row=0, column=i, sticky='ew', pady=2)

    Label(frame, text="Book Title", font=('Segoe UI', 9)).grid(row=1, column=0, sticky='w',
pady=5)
    book=Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    book.grid(row=1, column=1, sticky='ew', pady=5, padx=(5,0))

    Label(frame, text="Student ID", font=('Segoe UI', 9)).grid(row=2, column=0, sticky='w',
pady=5)
    student=Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    student.grid(row=2, column=1, sticky='ew', pady=5, padx=(5,0))

    Label(frame, text="Reward / Message", font=('Segoe UI', 9)).grid(row=3, column=0,
sticky='w', pady=5)
    reward=Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    reward.grid(row=3, column=1, sticky='ew', pady=5, padx=(5,0))

    frame.columnconfigure(1, weight=1)

    style = create_style()
    ttk.Button(frame, text="Post", style='Custom.TButton', command=submit).grid(row=4,
column=0, columnspan=2, pady=20)

def view_bounties():
    win = Toplevel(root)
    win.title("Bounties")
```

```
    win.geometry("1000x400")
    win.configure(bg="#F4F6F9")

    tree = ttk.Treeview(win,
columns=("ID","Book","Requester","Reward","Status","ClaimedBy"),
show="headings")

    for col in tree["columns"]:
        tree.heading(col, text=col)
        tree.column(col, width=150)

    tree.pack(fill=BOTH, expand=True)

    cur.execute("SELECT * FROM bounties")
    for b in cur.fetchall():
        tree.insert("", END, values=b)


# CLAIM BOUNTY
def claim_bounty():
    def submit():
        try:
            cur.execute("""
            UPDATE bounties
            SET status='Claimed', claimed_by=?
            WHERE bounty_id=? AND status='Open'
            """,(int(student.get()),int(bid.get())))

            if cur.rowcount==0:
                messagebox.showwarning("Error","Already Claimed / Invalid")
            else:
```

```
            conn.commit()
            messagebox.showinfo("Success","Bounty Claimed")

        win.destroy()
    except Exception as e:
        messagebox.showerror("Error",e)

win=Toplevel(root)
win.title("Claim Bounty")
win.geometry("400x250")
win.resizable(False, False)
win.transient(root)
win.grab_set()

frame = Frame(win, padx=20, pady=20)
frame.pack(expand=True, fill=BOTH)

headers = ["Field", "Value"]
for i, header in enumerate(headers):
    Label(frame, text=header, font=('Segoe UI', 10, 'bold'), bg='#f0f0f0', relief='solid',
bd=1, padx=10, pady=8, width=15).grid(row=0, column=i, sticky='ew', pady=2)

Label(frame, text="Bounty ID", font=('Segoe UI', 9)).grid(row=1, column=0, sticky='w',
pady=10)
bid=Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
bid.grid(row=1, column=1, sticky='ew', pady=10, padx=(5,0))

Label(frame, text="Your Student ID", font=('Segoe UI', 9)).grid(row=2, column=0,
sticky='w', pady=10)
student=Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
student.grid(row=2, column=1, sticky='ew', pady=10, padx=(5,0))
```

```
    frame.columnconfigure(1, weight=1)

    style = create_style()
    ttk.Button(frame, text="Claim", style='Custom.TButton',
command=submit).grid(row=3, column=0, columnspan=2, pady=20)

# ---------------- VIEW ISSUE RECORDS ----------------
def view_issues():
    win = Toplevel(root)
    win.title("Issue Records")
    win.geometry("1100x500")
    win.configure(bg="#F4F6F9")

    tree = ttk.Treeview(win,
columns=("IssueID","Book","Student","Issue","Due","Return","Status"),
show="headings")

    for col in tree["columns"]:
        tree.heading(col, text=col)
        tree.column(col, width=150)

    tree.pack(fill=BOTH, expand=True)

    cur.execute("""
    SELECT issues.issue_id, books.title, students.name,
           issues.issue_date, issues.due_date, issues.return_date, issues.status
    FROM issues
    JOIN books ON books.book_id = issues.book_id
    JOIN students ON students.student_id = issues.student_id
    """)
```

```
    for r in cur.fetchall():
        tree.insert("", END, values=r)

# ---------------- POLICY MANAGEMENT ----------------
def manage_policies():
    def submit():
        try:
            cur.execute("INSERT OR REPLACE INTO policies VALUES (?,?,?,?)",
                        (category.get(), int(max_books.get()), int(issue_days.get()),
float(fine_rate.get())))
            conn.commit()
            messagebox.showinfo("Success", "Policy updated")
            win.destroy()
        except Exception as e:
            messagebox.showerror("Error", e)

    win = Toplevel(root)
    win.title("Manage Policies")
    win.geometry("400x350")
    win.resizable(False, False)
    win.transient(root)
    win.grab_set()

    frame = Frame(win, padx=20, pady=20)
    frame.pack(expand=True, fill=BOTH)

    headers = ["Field", "Value"]
    for i, header in enumerate(headers):
        Label(frame, text=header, font=('Segoe UI', 10, 'bold'), bg='#f0f0f0', relief='solid',
bd=1, padx=10, pady=8, width=15).grid(row=0, column=i, sticky='ew', pady=2)
```

```
    Label(frame, text="Category", font=('Segoe UI', 9)).grid(row=1, column=0, sticky='w',
pady=5)
    category = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    category.grid(row=1, column=1, sticky='ew', pady=5, padx=(5,0))

    Label(frame, text="Max Books", font=('Segoe UI', 9)).grid(row=2, column=0,
sticky='w', pady=5)
    max_books = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    max_books.grid(row=2, column=1, sticky='ew', pady=5, padx=(5,0))

    Label(frame, text="Issue Days", font=('Segoe UI', 9)).grid(row=3, column=0, sticky='w',
pady=5)
    issue_days = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    issue_days.grid(row=3, column=1, sticky='ew', pady=5, padx=(5,0))

    Label(frame, text="Fine Rate", font=('Segoe UI', 9)).grid(row=4, column=0, sticky='w',
pady=5)
    fine_rate = Entry(frame, font=('Segoe UI', 9), relief='solid', bd=1)
    fine_rate.grid(row=4, column=1, sticky='ew', pady=5, padx=(5,0))

    frame.columnconfigure(1, weight=1)

    style = create_style()
    ttk.Button(frame, text="Update Policy", style='Custom.TButton',
command=submit).grid(row=5, column=0, columnspan=2, pady=20)

# ---------------- TOP NAVBAR ----------------
topbar = Frame(root, bg="#0B3C5D", height=60)
topbar.pack(side=TOP, fill=X)
```

```
Label(
    topbar,
    text="Library Management with Bounty System",
    bg="#0B3C5D",
    fg="white",
    font=("Segoe UI", 16, "bold")
).pack(expand=True, padx=20)

# ---------------- SIDEBAR ----------------
sidebar = Frame(root, bg="#08324A", width=220)
sidebar.pack(side=LEFT, fill=Y)

# --- Admin Section ---
admin_frame = Frame(sidebar, bg="#08324A")
admin_frame.pack(pady=20)

Label(admin_frame, text="Welcome to", fg="white", bg="#08324A", font=("Segoe UI",
10)).pack()

Label(admin_frame, text="Library", fg="white", bg="#08324A", font=("Segoe UI", 14,
"bold")).pack(pady=5)
def sidebar_button(text, command):
    Button(
        sidebar,
        text=text,
        command=command,
        bg="#0E4A6C",
        fg="white",
        relief="flat",
        anchor="w",
        font=("Segoe UI", 10),
```

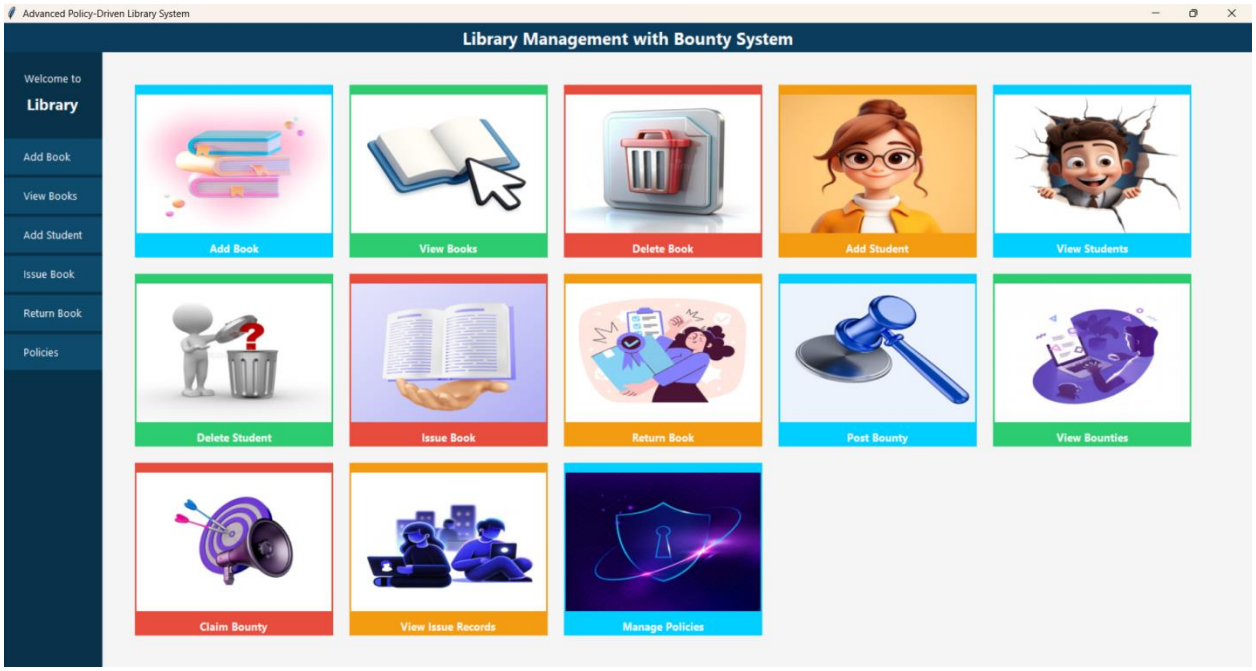
```
        padx=20,
        pady=8
    ).pack(fill=X, pady=2)

sidebar_button("Add Book", add_book)
sidebar_button("View Books", view_books)
sidebar_button("Add Student", add_student)
sidebar_button("Issue Book", issue_book)
sidebar_button("Return Book", return_book)
sidebar_button("Policies", manage_policies)

# ---------------- MAIN CONTENT AREA ----------------
main_area = Frame(root, bg="#F5F5F5")
main_area.pack(side=LEFT, fill=BOTH, expand=True)
# ---------------- LOAD ICON FUNCTION ----------------
icon_images = {}

def load_icon(path):
    img = Image.open(path)
    img = img.resize((250, 260))
    return ImageTk.PhotoImage(img)
# ---------------- DASHBOARD / TILE GRID ----------------
dashboard_frame = Frame(main_area, bg="#F5F5F5")
dashboard_frame.pack(padx=30, pady=30, fill=BOTH, expand=True)



# Define tile info (button text + function + color)
from PIL import Image, ImageTk

tiles = [
    ("Add Book", add_book, "#00CFFF", "icons/add_book.png"),
```

```
    ("View Books", view_books, "#2ECC71", "icons/view_books.png"),
    ("Delete Book", delete_book, "#E74C3C", "icons/delete_book.png"),
    ("Add Student", add_student, "#F39C12", "icons/add_student.png"),
    ("View Students", view_students, "#00CFFF", "icons/view_students.png"),
    ("Delete Student", delete_student, "#2ECC71", "icons/delete_student.png"),
    ("Issue Book", issue_book, "#E74C3C", "icons/issue.png"),
    ("Return Book", return_book, "#F39C12", "icons/return.png"),
    ("Post Bounty", post_bounty, "#00CFFF", "icons/post_bounty.png"),
    ("View Bounties", view_bounties, "#2ECC71", "icons/view_bounty.png"),
    ("Claim Bounty", claim_bounty, "#E74C3C", "icons/claim.png"),
    ("View Issue Records", view_issues, "#F39C12", "icons/records.png"),
    ("Manage Policies", manage_policies, "#00CFFF", "icons/policy.png")
]

# Keep references so images don't get garbage collected
tile_images = []

rows = 3
cols = 5

for idx, (text, func, color, icon_path) in enumerate(tiles):
    r = idx // cols
    c = idx % cols

    tile = Frame(dashboard_frame, bg=color, width=220, height=180)
    tile.grid_propagate(False)
    tile.grid(row=r, column=c, padx=10, pady=10, sticky="nsew")

    # Load icon if it exists
    if os.path.exists(icon_path):
        img = Image.open(icon_path)
```

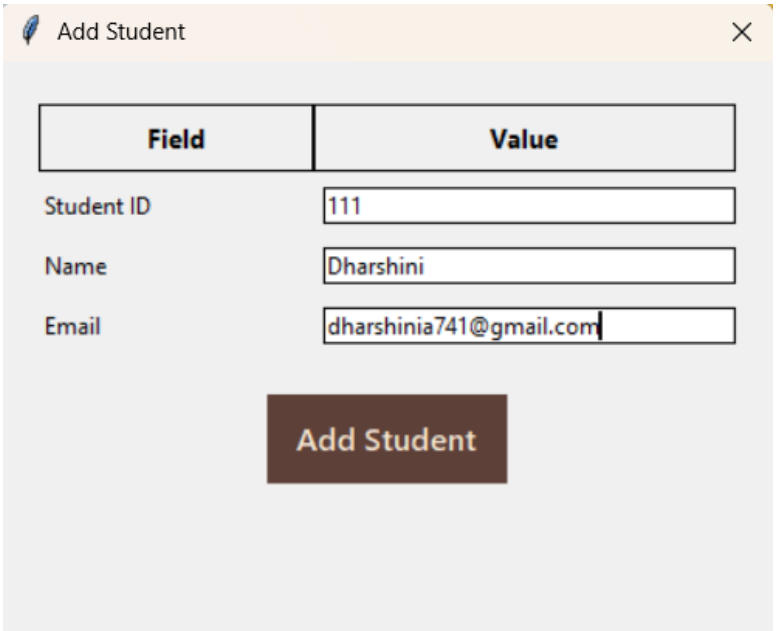
```
        img = img.resize((240, 170))  # resize to fit tile
        photo = ImageTk.PhotoImage(img)
        tile_images.append(photo)  # keep reference
        lbl_img = Label(tile, image=photo, bg=color)
        lbl_img.pack(pady=(10,5))
    else:
        lbl_img = None

    lbl_text = Label(tile, text=text, bg=color, fg="white", font=("Segoe UI", 10, "bold"))
    lbl_text.pack()

    tile.bind("<Button-1>", lambda e, f=func: f())
    lbl_text.bind("<Button-1>", lambda e, f=func: f())
    if lbl_img:
        lbl_img.bind("<Button-1>", lambda e, f=func: f())


root.mainloop()
conn.close()
    frame.pack(expand=True, fill=BOTH)

    headers = ["Field", "Value"]
    for i, header in enumerate(headers):
        Label(frame, text=header, font=('Segoe UI', 10, 'bold'), bg='#f0f0f0', relief='solid',
bd=1, padx=10, pady=8, width=15).grid(row=0, column=i, sticky='ew', pady=2)
```

D. SAMPLE INPUT

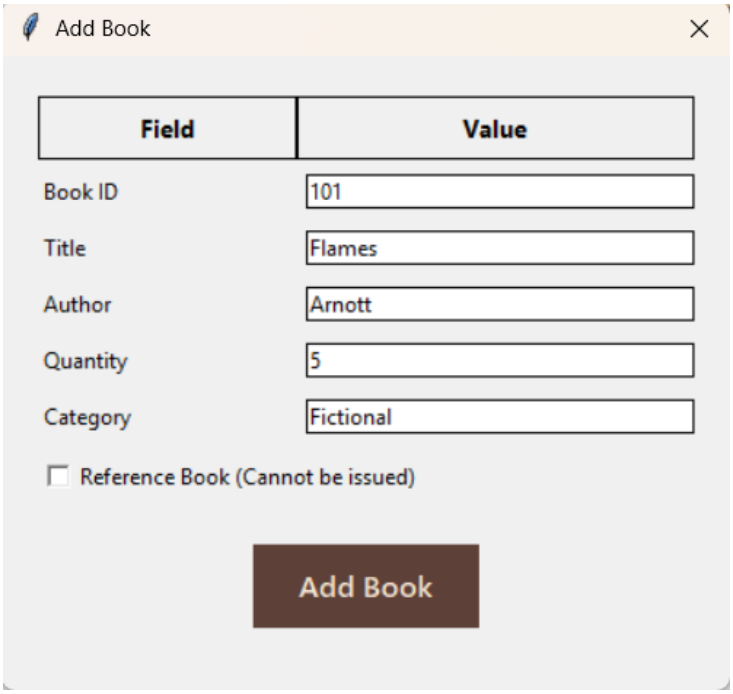
CATALOG



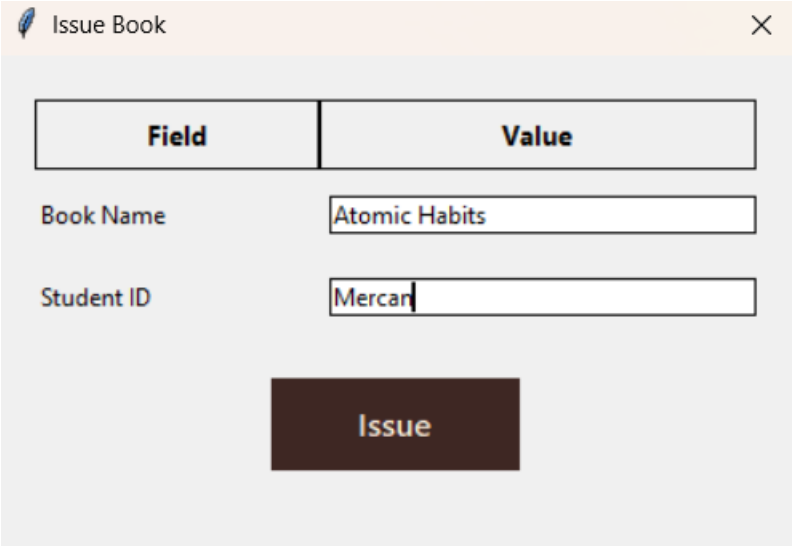
ADD STUDENT FORM



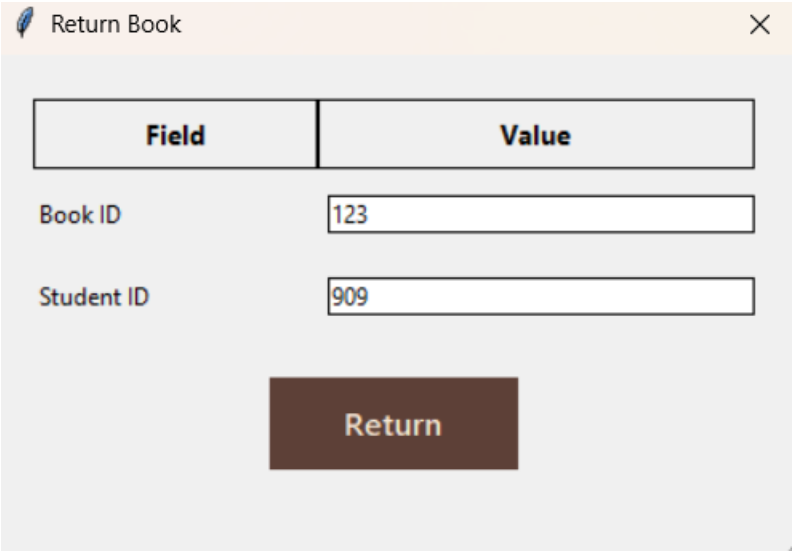
ADD BOOK FORM



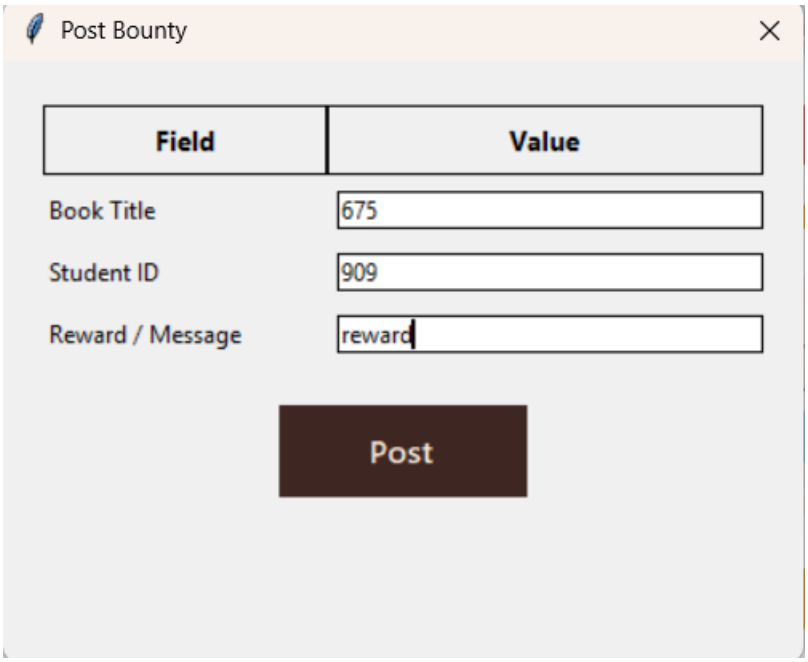
ISSUE BOOK FORM



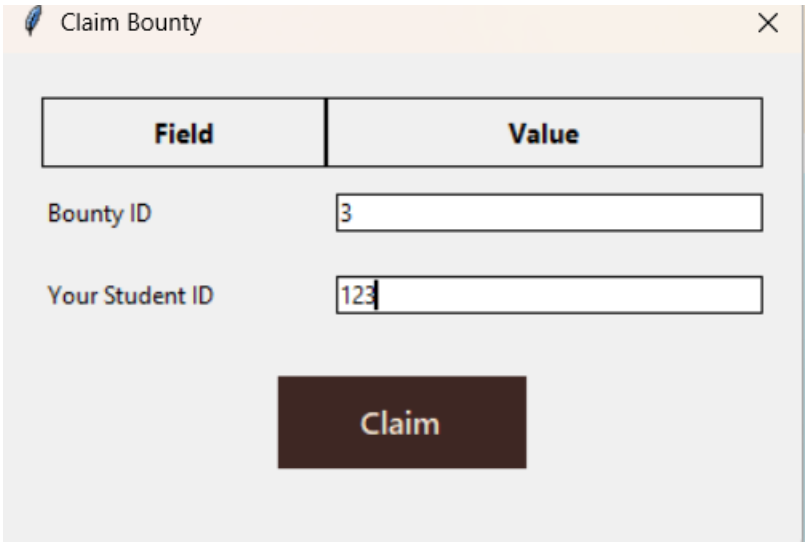
RETURN BOOK FORM



POST BOUNTY FORM



CLAIM BOUNTY FORM



E. SAMPLE OUTPUT:

VIEW BOOK FORM

Books

ID	Title	Author	Qty	Category	Reference
98	Obama2	Obama	5	History	No
101	Flames	Arnott	5	Fictional	No
102	Beyond peace	Richard	10	Self development	No
103	Gathering stories	Winsston	20	Story	No
104	To Kill a Mockingbird	Harper Lee	17	Fiction	No
105	1984	George Orwell	20	Dystopian	No
106	Pride and Prejudice	Jane Austen	20	Romance	No
123	Atomic Habits	James Clear	4	Self Development	No
345	Almanack of Ravikanth	Ravi	6	Self Development	Yes
567	Bride	Michale	9	Romantic	No
632	Atom	James	5	science	No
888	Obama	Obama	5	History	No

VIEW STUDENT FORM

Students

ID	Name	Email
111	Dharshini	dharshinia741@gmail.com
201	devayani	m1998devayani@gmail.com
417	Clara	Clarara@gmail.com
639	Emin	Eminarafta@gmail.com
909	Mercan	dharshinike20@gmail.com

VIEW BOUNTY FORM

Bounties

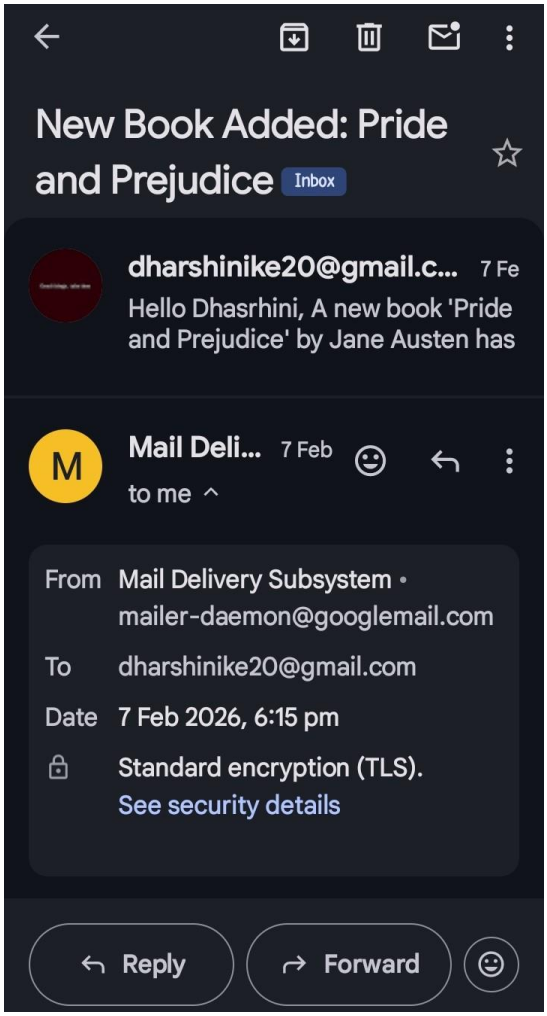
ID	Book	Requester	Reward	Status	ClaimedBy
1	Obama	417	Reward	Claimed	909
2	1984	201	Message	Claimed	639
3	Atom	528	Reward	Open	None

ISSUE RECORD FORM

Issue Records

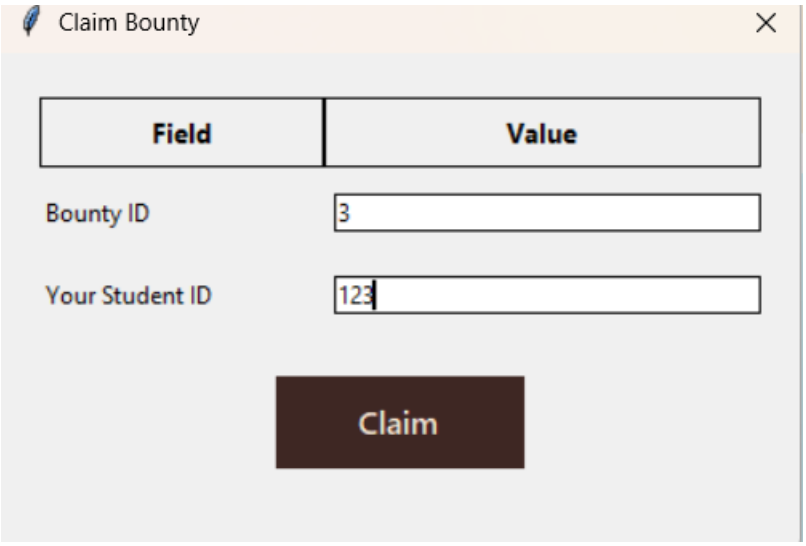
IssueID	Book	Student	Issue	Due	Return	Status
2	Bride	Clara	2026-02-03	2026-02-23	2026-02-03	Returned
3	Atomic Habits	Mercan	2026-02-07	2026-02-27	2026-02-07	Returned
4	Bride	Mercan	2026-02-07	2026-02-27	2026-02-07	Returned
5	Bride	Mercan	2026-02-07	2026-02-27	2026-02-16	Returned
6	To Kill a Mockingbird	devayani	2026-02-07	2026-02-17	None	Not Returned
8	To Kill a Mockingbird	Emin	2026-02-07	2026-02-17	2026-02-13	Returned
9	To Kill a Mockingbird	Clara	2026-02-07	2026-02-17	None	Not Returned

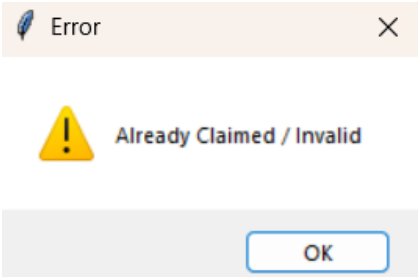
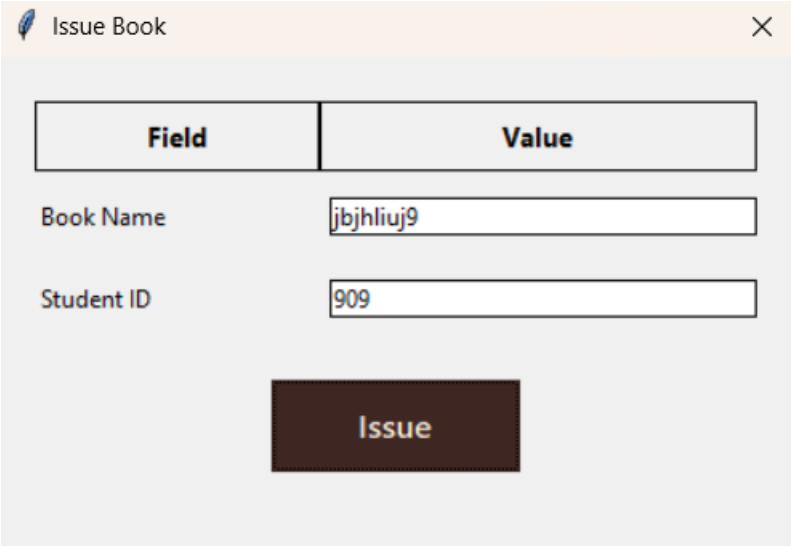
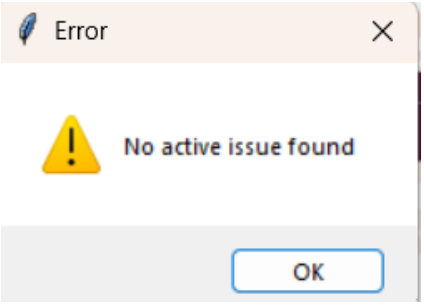
MAIL NOTIFICATION



ERROR POP UP

Wrong Claim ID





Wrong Issue Detail